

MANUAL TECNICO

Kevin Steven Uribe Lara

Análisis y Desarrollo de Software

SENA

GA10-220501097-AA10-EV01

4 de Julio de 2026

INTRODUCCIÓN

El presente manual técnico documenta la arquitectura, la base de datos, los estándares y el procedimiento de instalación de mi sistema BookSync, el cual es un sistema de tipo ERP para la gestión bibliotecaria. Permite a los usuarios finales consultar el catálogo, reservar y consultar sus préstamos, mientras que el rol administrador gestiona el inventario de libros, confirma o rechaza reservas y controla el ciclo completo de préstamo y devolución.

ALCANCE

Este manual cubre la descripción de los componentes de software y de base de datos que conforman a BookSync, los prerequisites y pasos necesarios para instalar el sistema en un entorno local o desplegarlo en los servicios en la nube utilizados en producción, así como la documentación de la estructura del código fuente del proyecto.

PRERREQUISITOS DE INSTALACIÓN DEL SISTEMA

1. Requisitos de software

- Node.js versión 18 o superior (incluye npm) para ejecutar el backend y compilar el frontend.
- MySQL Server versión 8.0 o superior.
- Git, para clonar el repositorio de código fuente.
- Un editor de código (VS Code recomendado) y un cliente de administración de bases de datos (MySQL Workbench, DBeaver o similar).
- Navegador web moderno con soporte de ES2020+ (Chrome, Edge, Firefox).

2. Requisitos de hardware (mínimos recomendados para desarrollo/pruebas)

- Procesador de doble núcleo, 2.0 GHz o superior.
- 4 GB de memoria RAM (8 GB recomendado).
- 2 GB de espacio libre en disco para dependencias de Node.js y el motor de base de datos.
- Conexión a Internet para la instalación de dependencias (npm) y para el uso de los servicios en la nube de producción.

3. Variables de entorno requeridas (backend)

El backend requiere un archivo .env ubicado en la carpeta Server con las siguientes variables:

```
PORT=3000

DB_HOST=localhost

DB_USER=root

DB_PASSWORD=tu_password

DB_NAME=booksync

JWT_SECRET=tu_clave_secreta

JWT_EXPIRES_IN=8h

EMAIL_USER=tu_correo@gmail.com

EMAIL_PASS=tu_app_password
```

4. Cuentas y servicios externos (ambiente de producción)

- Cuenta en Azure para el despliegue del frontend en Azure Static Web Apps.
- Cuenta en Render para el despliegue del backend (API REST).
- Cuenta en Railway para el servicio administrado de MySQL en producción.
- Repositorio en GitHub (github.com/kevin-lara-dev/Booksync) conectado a los servicios de despliegue continuo.

FRAMEWORKS Y ESTANDARES CON LOS QUE SE CONSTRUYO EL SISTEMA

1. Frontend

- React 19, para la construcción de la interfaz como una aplicación de página única (SPA).
- Vite, como herramienta de compilación y servidor de desarrollo.
- React Router DOM v7, para el enrutamiento entre vistas y la protección de rutas (PrivateRoute / AdminRoute).
- Axios, con interceptor para el manejo del token JWT y cierre de sesión automático ante respuestas 401.
- CSS modular por vista, SweetAlert2 para alertas y FontAwesome para iconografía.

2. Backend

- Node.js con Express 5, para la construcción de la API REST.
- mysql2, con pool de conexiones para el acceso a la base de datos (sin ORM; las consultas SQL se definen explícitamente en la capa de modelos).
- jsonwebtoken (JWT) y bcryptjs, para autenticación y cifrado de contraseñas.
- dotenv, para la gestión de variables de entorno; cors, para el control de acceso entre dominios; nodemon, como utilidad de desarrollo.

3. Base de datos

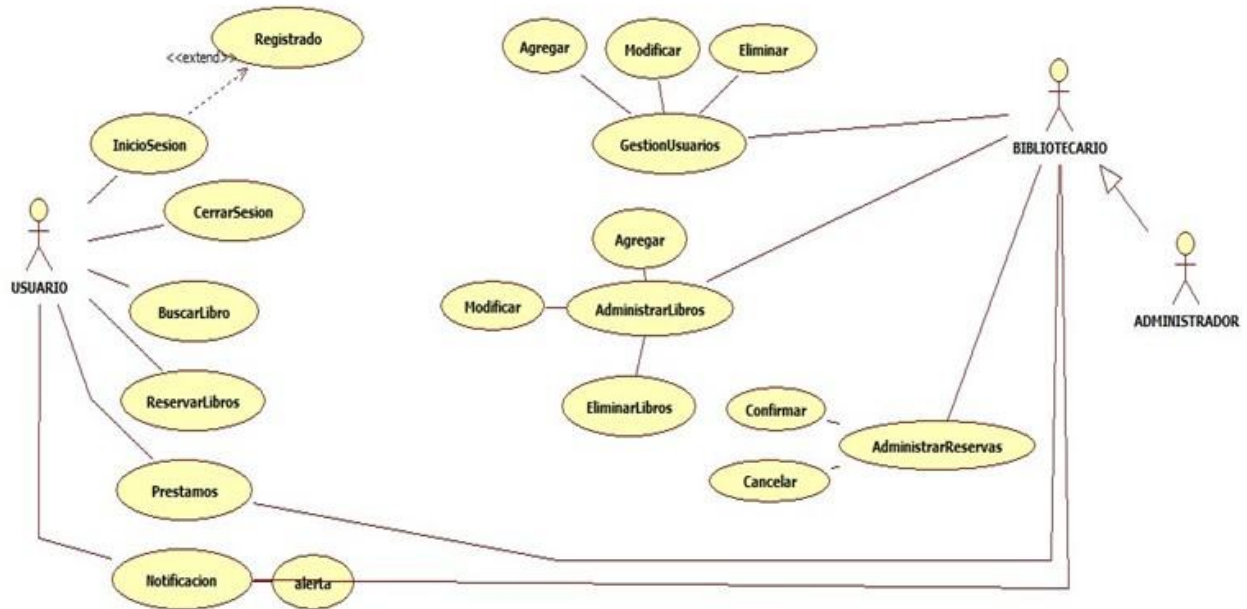
- MySQL 8, motor de almacenamiento InnoDB, juego de caracteres utf8mb4.

4. Estándares y buenas prácticas aplicadas

- Arquitectura en capas de tipo MVC en el backend: routes → middlewares → controllers → models.
- API de tipo REST, con verbos HTTP estándar (GET, POST, PUT, PATCH, DELETE) y formato de intercambio JSON.
- Autenticación basada en JWT (estándar RFC 7519) y control de acceso basado en roles (RBAC): administrador, bibliotecario, usuario.
- Control de versiones con Git y GitHub, bajo la rama principal main.
- Convenciones de nomenclatura: camelCase para variables y funciones en JavaScript; snake_case para tablas y columnas de la base de datos.
- Despliegue continuo (CI/CD) mediante flujos de trabajo de GitHub Actions hacia Azure Static Web Apps.

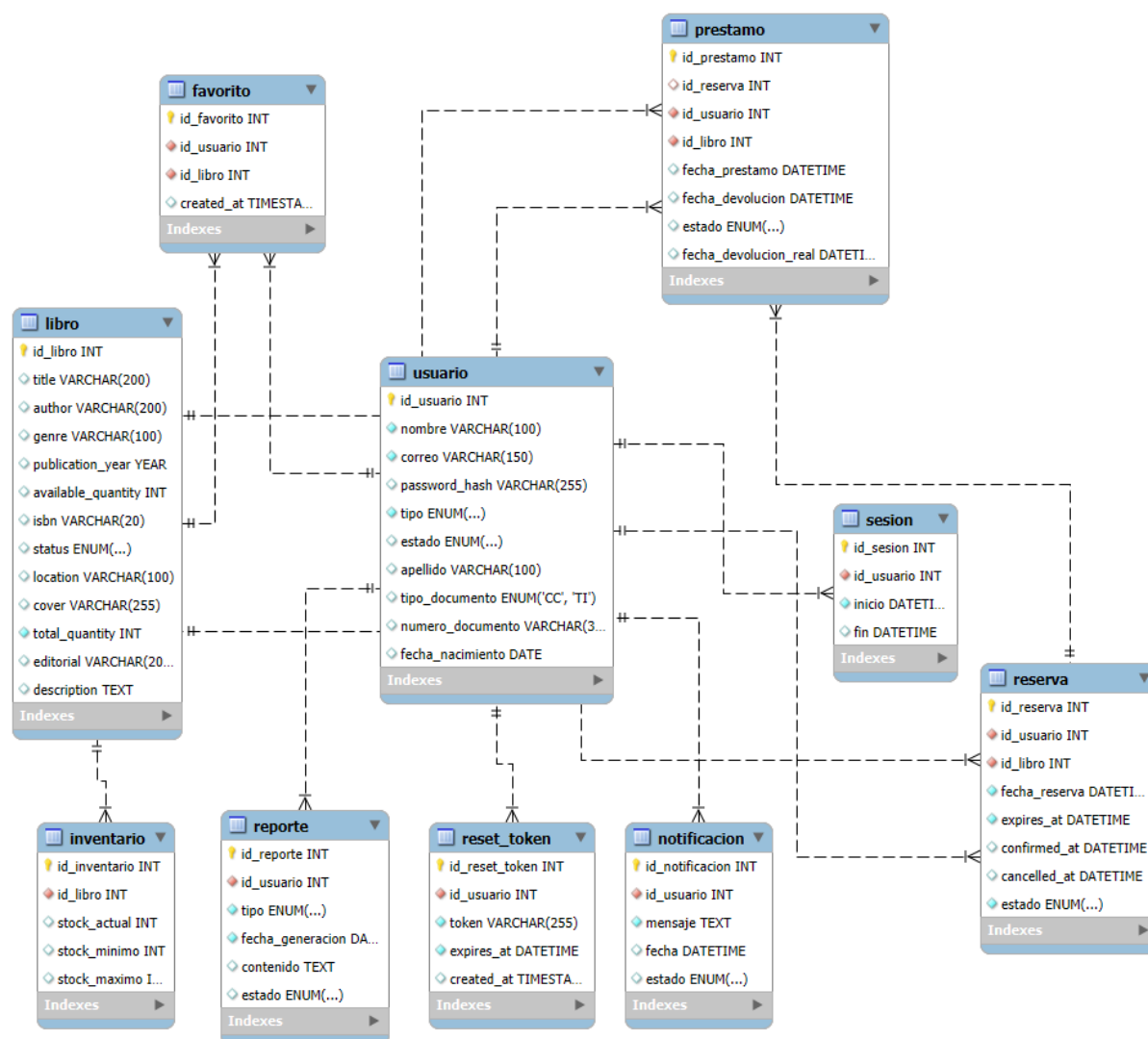
DIAGRAMA DE CASOS DE USO DEL SISTEMA

El sistema define tres actores principales: el Usuario (lector de la biblioteca), el bibliotecario (empleado encargado de la administración de préstamos, inventario y reservas) y el Administrador (encargado de la gestión completa del sistema).



MODELO ENTIDAD-RELACIÓN DE LA BASE DE DATOS

La base de datos booksync está compuesta por 10 tablas relacionadas entre sí. La entidad usuario es el eje central del modelo: se relaciona con reserva, préstamo, favorito, notificación, reporte, sesion y reset_token. La entidad libro se relaciona con reserva, préstamo, favorito e inventario (relación uno a uno). La tabla préstamo puede opcionalmente originarse desde una reserva confirmada.



DICCIONARIO DE DATOS — BOOKSYNC

Base de datos BookSync · 10 tablas · 68 campos

Leyenda: PK = Llave primaria · FK / Índice = Llave foránea o índice (MUL) · UNIQUE = Valor único · — = Sin valor por defecto

TABLA: FAVORITO (4 campos)

Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_favorito	int unsigned	NO	PK	—	AUTO_INCREMENT
id_usuario	int unsigned	NO	FK / Índice	—	
id_libro	int unsigned	NO	FK / Índice	—	
created_at	timestamp	SÍ		CURRENT_TIMESTAMP	DEFAULT GENERATED

TABLA: INVENTARIO (5 campos)

Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_inventario	int unsigned	NO	PK	—	AUTO_INCREMENT
id_libro	int unsigned	NO	FK / Índice	—	
stock_actual	int unsigned	SÍ		—	
stock_minimo	int unsigned	SÍ		—	
stock_maximo	int unsigned	SÍ		—	

TABLA: LIBRO (13 campos)

Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_libro	int unsigned	NO	PK	—	AUTO_INCREMENT
title	varchar(200)	SÍ		—	
author	varchar(200)	SÍ		—	
genre	varchar(100)	SÍ		—	
publication_year	year	SÍ		—	
available_quantity	int unsigned	SÍ		—	
isbn	varchar(20)	SÍ	UNIQUE	—	
status	enum('disponible','pres tado','dañado','inactivo)	SÍ		—	
location	varchar(100)	SÍ		—	
cover	varchar(255)	SÍ		—	
total_quantity	int	NO		0	
editorial	varchar(200)	SÍ		—	
description	text	SÍ		—	

TABLA: NOTIFICACION (5 campos)

Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_notificacion	int unsigned	NO	PK	—	AUTO_INCREMENT
id_usuario	int unsigned	NO	FK / Índice	—	
mensaje	text	NO		—	

fecha	datetime	SÍ		CURRENT_TIMESTAMP	DEFAULT GENERATED
estado	enum('pendiente','leida')	SÍ		pendiente	

TABLA: PRESTAMO (8 campos)					
Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_prestamo	int unsigned	NO	PK	—	AUTO_INCREMENT
id_reserva	int unsigned	SÍ	FK / Índice	—	
id_usuario	int unsigned	NO	FK / Índice	—	
id_libro	int unsigned	NO	FK / Índice	—	
fecha_prestamo	datetime	SÍ		CURRENT_TIMESTAMP	DEFAULT GENERATED
fecha_devolucion	datetime	SÍ		—	
estado	enum('activo','devuelto','vencido')	SÍ		activo	
fecha_devolucion_real	datetime	SÍ		—	

TABLA: REPORTE (6 campos)					
Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_reporte	int unsigned	NO	PK	—	AUTO_INCREMENT
id_usuario	int unsigned	NO	FK / Índice	—	
tipo	enum('mensual','personal','inventario')	NO		—	
fecha_generacion	date	NO		—	
contenido	text	SÍ		—	
estado	enum('activo','cerrado')	SÍ		activo	

TABLA: RESERVA (8 campos)					
Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_reserva	int unsigned	NO	PK	—	AUTO_INCREMENT
id_usuario	int unsigned	NO	FK / Índice	—	
id_libro	int unsigned	NO	FK / Índice	—	
fecha_reserva	datetime	NO		CURRENT_TIMESTAMP	DEFAULT GENERATED
expires_at	datetime	NO		—	
confirmed_at	datetime	SÍ		—	
cancelled_at	datetime	SÍ		—	
estado	enum('activa','confirmada','cancelada','expirada','prestada')	NO	FK / Índice	activa	

TABLA: RESET_TOKEN (5 campos)					
Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_reset_token	int unsigned	NO	PK	—	AUTO_INCREMENT
id_usuario	int unsigned	NO	FK / Índice	—	

token	varchar(255)	NO	UNIQUE	—	
expires_at	datetime	NO		—	
created_at	timestamp	SÍ		CURRENT_TIMESTAMP	DEFAULT GENERATED

TABLA: SESION (4 campos)					
Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_sesion	int unsigned	NO	PK	—	AUTO_INCREMENT
id_usuario	int unsigned	NO	FK / Índice	—	
inicio	datetime	NO		—	
fin	datetime	SÍ		—	

TABLA: USUARIO (10 campos)					
Campo	Tipo de dato	¿Permite NULL?	Llave	Valor por defecto	Extra
id_usuario	int unsigned	NO	PK	—	AUTO_INCREMENT
nombre	varchar(100)	NO		—	
correo	varchar(150)	NO	UNIQUE	—	
password_hash	varchar(255)	SÍ		—	
tipo	enum('administrador','bibliotecario','usuario')	NO		—	
estado	enum('activo','inactivo','bloqueado')	SÍ		—	
apellido	varchar(100)	SÍ		—	
tipo_documento	enum('CC','TI')	SÍ		—	
numero_documento	varchar(30)	SÍ		—	
fecha_nacimiento	date	SÍ		—	

SCRIPTS DE INSTALACIÓN

1. Clonar el repositorio

- git clone <https://github.com/kevin-lara-dev/Booksync.git>
- cd Booksync

2. Configurar y levantar el backend

- cd Server
- npm install
- Crear archivo .env con las credenciales de MySQL y JWT_SECRET
- Importar el esquema y los datos de la base de datos
- mysql -u root -p booksync < bd_booksync_final.sql

- Iniciar el servidor en modo desarrollo
- `npm run dev`
- Disponible en `http://localhost:3000`

3. Configurar y levantar el frontend

- `cd ../Client`
- `npm install`
- `npm run dev`
- Disponible en `http://localhost:5173`

4. Compilación para producción (frontend)

- `cd Client`
- `npm run build`
- Genera la carpeta `dist/` lista para desplegar en Azure Static Web Apps

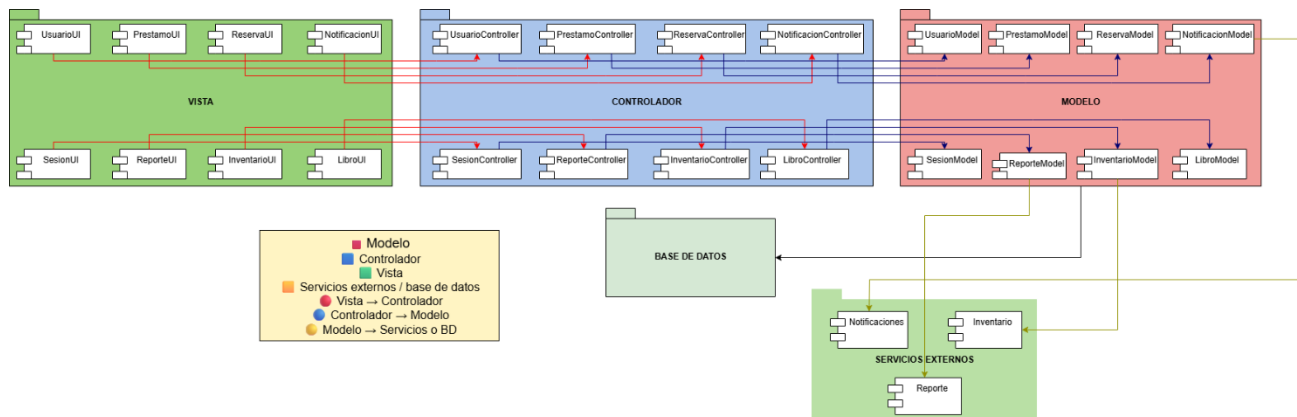
5. Despliegue en producción

El sistema se encuentra desplegado utilizando el siguiente esquema de servicios en la nube:

- **Frontend (Azure Static Web Apps):** se conecta al repositorio de GitHub y compila automáticamente la carpeta `Client` con cada cambio en la rama `main`. URL: **`https://red-bush-0947a890f.7.azurestaticapps.net`**
- **Backend (Render):** despliega la carpeta `Server` como Web Service de Node.js, ejecutando **`npm install`** y **`node src/server.js`**, con las variables de entorno configuradas en el panel de Render. URL: **`https://booksync-api-dnju.onrender.com`**
- **Base de datos (Railway):** instancia administrada de MySQL 8; el backend se conecta mediante las credenciales provistas por Railway en las variables **`DB_HOST`**, **`DB_USER`**, **`DB_PASSWORD`** y **`DB_NAME`**. El archivo `bd_booksync_final.sql` contiene la definición completa de las 10 tablas junto con sus llaves primarias, llaves foráneas, restricciones e índices, por lo que su importación deja la base de datos lista para el funcionamiento del backend.

DIAGRAMA DE COMPONENTES

BookSync sigue una arquitectura por capas basada en el patrón Modelo, Vista, Controlador (MVC). Cada módulo (Usuario, Préstamo, Reserva, Notificación, Sesión, Reporte, Inventario y Libro) tiene su propia Vista, Controlador y Modelo: la Vista consume al Controlador, el Controlador usa al Modelo, y el Modelo es el único que accede a la base de datos. Los módulos de Reporte, Inventario y Notificación además se apoyan en servicios externos. La comunicación fluye siempre en una sola dirección, manteniendo cada capa enfocada solo en su función.



Documentación del código fuente

El repositorio se organiza en dos proyectos independientes, Client (frontend) y Server (backend), como se muestra a continuación:

BOOKSYNC/

- ├── Client/ # Frontend React + Vite
 - ├── src/
 - ├── components/ # Sidebar, AdminRoute, PrivateRoute
 - ├── context/ # AuthContext (JWT, login, logout)
 - ├── hooks/ # useToast, useLogoutToast
 - ├── pages/
 - ├── auth/ # Login, Register, Help, ResetPassword, ForgotPassword, ForgotEmailSent
 - ├── dashboard/ # Home, Detalle, ReservasUsuario, PrestamosUsuarios, Account
 - ├── Admin/ # InventarioAdmin, ReservasAdmin, UsuariosAdmin, PrestamosAdmin
 - ├── services/ # Funciones Axios por módulo
 - ├── styles/ # CSS modular por vista

```

|
└─ Server/          # Backend Node.js + Express
    └─ src/
        └─ config/   # configuración con la Base de datos
        └─ controllers/ # Lógica de negocio por módulo
        └─ models/    # Consultas SQL con mysql2 (sin ORM)
        └─ routes/    # Endpoints REST por módulo
    └─ middlewares/  # auth.middleware, role.middleware

```

1. Módulos del backend

- **routes:** define los endpoints REST agrupados por dominio (auth, users, libros, reservas, prestamos) y los asocia a su controlador correspondiente.
- **middlewares:** auth.middleware valida el token JWT en cada petición protegida; role.middleware verifica que el rol del usuario autenticado tenga permiso sobre el recurso solicitado.
- **controllers:** contienen la lógica de negocio de cada módulo (validaciones, reglas de reserva/préstamo, control de stock, etc.).
- **models:** encapsulan las consultas SQL ejecutadas contra MySQL a través del pool de conexiones de mysql2.

2. Principales endpoints de la API

A continuación, se resumen los endpoints REST expuestos por el backend, organizados por módulo:

Endpoint	Acceso	Descripción
POST /auth/register	Público	Registro de un nuevo usuario.
POST /auth/login	Público	Autenticación de usuario y emisión del token JWT.
POST /auth/forgot-password	Público	Solicita el envío de un correo para restablecer la contraseña.
POST /auth/reset-password/:token	Público	Restablece la contraseña usando el token enviado por correo.

Endpoint	Acceso	Descripción
GET /libros	Autenticado	Consulta del catálogo de libros con filtros opcionales.
GET /libros/genres	Autenticado	Consulta de los géneros disponibles.
GET /libros/:id	Autenticado	Consulta del detalle de un libro.
POST /libros	Admin	Creación de un libro.
POST /libros/import	Admin	Importación masiva de libros desde un archivo CSV.
PUT /libros/:id	Admin	Edición parcial de un libro.
DELETE /libros/:id	Admin	Eliminación (soft delete) de un libro.
GET /favoritos	Autenticado	Consulta de mis favoritos con los datos del libro.
GET /favoritos/ids	Autenticado	Consulta de los IDs de mis libros favoritos.
GET /favoritos/:idLibro	Autenticado	Verifica si un libro está en mis favoritos.
POST /favoritos/:idLibro	Autenticado	Agrega un libro a mis favoritos.
DELETE /favoritos/:idLibro	Autenticado	Quita un libro de mis favoritos.
POST /reservas/:idLibro	Autenticado	Creación de una reserva para un libro.
GET /reservas/mis	Autenticado	Consulta de mis reservas.
DELETE /reservas/:idReserva	Autenticado	Cancelación de una reserva propia.
GET /reservas/admin	Admin	Consulta de todas las reservas del sistema.
PATCH /reservas/admin/:idReserva/confirmar	Admin	Confirmación de una reserva.
PATCH /reservas/admin/:idReserva/cancelar	Admin	Cancelación de una reserva desde el panel administrativo.
POST /prestamos/reserva/:idReserva	Admin	Registro de un préstamo a partir de una reserva confirmada.
PATCH /prestamos/devolver/:idPréstamo	Admin	Registro de la devolución de un préstamo.

Endpoint	Acceso	Descripción
GET /prestamos/list	Admin	Consulta de todos los préstamos del sistema.
GET /prestamos/MisPrestamos	Autenticado	Consulta de mi historial de préstamos.
GET /users	Admin	Listado de todos los usuarios del sistema.
GET /users/profile	Autenticado	Consulta de mi perfil.
PUT /users/profile	Autenticado	Actualización de mis datos de perfil.
PATCH /users/profile/password	Autenticado	Cambio de mi contraseña.
DELETE /users/profile	Autenticado	Desactivación de mi cuenta (soft delete).
PATCH /users/:id/status	Admin	Activación o desactivación de un usuario.
PATCH /users/:id/role	Admin	Cambio de rol de un usuario.

3. Seguridad implementada en el código

- Las contraseñas nunca se almacenan en texto plano: se cifran con **bcryptjs** antes de guardarse en la columna **password_hash**.
- Cada ruta protegida valida el **token JWT** enviado en el encabezado **Authorization** y, cuando aplica, el rol del usuario autenticado.
- El interceptor de Axios en el frontend cierra la sesión automáticamente cuando el backend responde con estado HTTP 401 (token vencido o inválido).
- Las reservas no confirmadas expiran automáticamente de acuerdo con la columna **expires_at** de la tabla reserva.